

Curso Profesional de JasperReports 6.20.0 Community

Módulo 6 — Exportación

TEORÍA

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Módulo 6 — Teoría de exportación

Proyecto acumulativo: **EditorialReports**. Fuente original preservada en `.github/source/M6_ORIGINAL.md`.

La teoría conserva los objetivos del material original y corrige las APIs que no corresponden a JasperReports Library 6.20.0. Cada corrección se traza a un checkpoint compilado y ejecutado.

Punto 6.1 — Exportación a PDF

Objetivos de aprendizaje

- Comprender el papel del exportador PDF y su ubicación en la biblioteca.
- Diferenciar los métodos simples de `JasperExportManager` de los exportadores avanzados.
- Configurar las propiedades del exportador mediante `SimplePdfExporterConfiguration`.
- Establecer metadatos del documento PDF (título, autor, palabras clave).
- Aplicar protección con contraseña y permisos al PDF generado.
- Documentar la exportación a PDF del proyecto `EditorialReports`.

Bloque 1 — De JasperPrint al PDF: exportación simple y avanzada

El punto de partida no cambia respecto a M5: `informe_ventas.jrxml` se compila, se llena con `SQLite` y produce un `JasperPrint` de seis páginas. El Módulo 6 comienza después del llenado. Esto es importante porque el formato PDF no se diseña en un JRXML diferente: el mismo documento en memoria puede enviarse a varios exportadores. JasperReports ofrece una vía simple, `JasperExportManager.exportReportToPdfFile`, y una vía avanzada basada en `JRPdfExporter`. La primera es suficiente cuando basta con escribir un PDF sin personalización; la segunda separa entrada, salida y configuración y permite controlar metadatos, compresión y seguridad.

JAVA

```
JRPdfExporter exportador = new JRPdfExporter();
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(rutaPdf));
exportador.exportReport();
```

`SimpleExporterInput` adapta el `JasperPrint` a la entrada esperada por el exportador. `SimpleOutputStreamExporterOutput` define el destino binario y `exportReport()` realiza la conversión. El checkpoint 6.1 conserva también la compilación del subreporte antes del maestro, la conexión heredada y todos los parámetros de M5; sólo sustituye la exportación simple de ventas por la ruta avanzada.

El E2E verifica que el archivo principal empieza por `%PDF-`, que `JasperPrint` sigue teniendo seis páginas y que todos los informes heredados continúan generándose. Por tanto, el cambio está integrado en el proyecto acumulativo completo.

Bloque 2 — Metadatos correctos con `SimplePdfExporterConfiguration`

`SimplePdfExporterConfiguration` configura propiedades que afectan al documento PDF completo. En JasperReports 6.20.0 los metadatos se establecen con `setMetadataTitle`, `setMetadataAuthor`, `setMetadataSubject`, `setMetadataKeywords` y `setMetadataCreator`. El material original empleaba nombres genéricos como `setTitle` y `setAuthor`; el checkpoint corrige esa diferencia sin alterar el objetivo pedagógico.

JAVA

```
SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");
configuracion.setMetadataAuthor("Departamento Comercial");
configuracion.setMetadataSubject("Resumen de ventas del catálogo");
configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");
configuracion.setMetadataCreator("JasperReports 6.20.0");
configuracion.setDisplayMetadataTitle(Boolean.TRUE);
configuracion.setCompressed(Boolean.TRUE);
exportador.setConfiguration(configuracion);
```

Los metadatos son propiedades del fichero, no elementos visuales de la página. Por eso no modifican bandas, estilos ni el número de páginas. `setDisplayMetadataTitle(Boolean.TRUE)` permite que los lectores que respetan esa preferencia utilicen el título documental. `setCompressed(Boolean.TRUE)` activa la compresión del PDF.

La fuente original proponía aplicar `setCharacterEncoding` directamente sobre la configuración PDF. Ese método no forma parte de `SimplePdfExporterConfiguration` en esta versión y no se reproduce. La corrección queda probada por la compilación Maven y por `pdftinfo`, que comprueba título, autor y creador en el archivo generado.

Bloque 3 — Cifrado, contraseñas y permisos

Para que el alumno pueda comprobar simultáneamente un PDF normal con metadatos y otro protegido, el checkpoint genera `informe_ventas.pdf` e `informe_ventas_protegido.pdf`. El segundo activa cifrado y separa contraseña de usuario y de propietario.

JAVA

```
SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
configuracion.setEncrypted(Boolean.TRUE);
configuracion.setUserPassword("editorial2026");
configuracion.setOwnerPassword("editorial-admin");
configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
```

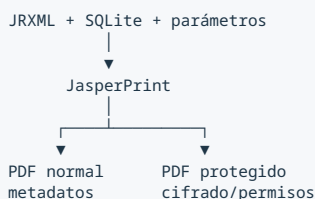
`setUserPassword` define la contraseña necesaria para abrir el documento. `setOwnerPassword` protege las operaciones reservadas al propietario. `setAllowedPermissionsHint` expresa los permisos concedidos. La fuente proponía `setPdfPassword(usuario, propietario)` y un `EnumSet<PdfPermissionsEnum>`; la práctica conserva el objetivo de seguridad pero usa la API que realmente compila en 6.20.0.

El E2E ejecuta `pdftinfo -upw editorial2026` sobre el PDF protegido. Si la contraseña descrita por la práctica no abre el fichero, el checkpoint falla. Esto vincula la explicación directamente al comportamiento producido.

Bloque 4 — Entrada, salida y reutilización del mismo JasperPrint

El exportador avanzado sigue el patrón común de JasperReports: una entrada, una salida y una configuración. El `JasperPrint` no se vuelve a llenar para cada formato. Esto permite que, a partir de 6.2, el mismo documento se entregue también a XLSX, HTML, CSV, XML y RTF.

TEXT



Separar llenado y exportación reduce trabajo y mantiene coherencia: todos los formatos representan exactamente el mismo estado de datos. Si `JasperFillManager.fillReport` falla, ninguno de los exportadores se ejecuta. Si sólo falla `JRPdfExporter`, el problema está en la configuración o en la salida PDF.

La ruta SQLite se mantiene como `jdbc:sqlite:../EditorialReportsJava/data/editorial.db` porque el generador se ejecuta desde `EditorialReports`. También se mantiene `System.exit(1)` en el catch para que un fallo sea visible en CI.

Bloque 5 — Evidencia reproducible y documentación

Una práctica de exportación sólo está terminada cuando el archivo generado puede inspeccionarse. El checkpoint 6.1 comprueba compilación Java, compilación JRXML, llenado del documento y validación del PDF. A ello se añaden los contratos heredados: 14 libros, 9 ventas, 31 unidades y 633,40 €.

TEXT

```
M5/5.6
├── M6/6.1
│   ├── mismo informe_ventas.jrxml
│   ├── mismo EditorialStyles.jrtx
│   ├── GeneradorInformeVentas.java modificado
│   ├── EXPORTACION_PDF.md nuevo
│   ├── output/informe_ventas.pdf
│   └── output/informe_ventas_protegido.pdf
```

La auditoría de trazabilidad exige que ningún archivo heredado del proyecto de informes desaparezca o cambie. M6 altera la capa Java de exportación y añade documentación/salidas, pero no reescribe el diseño validado de M5.

`EXPORTACION_PDF.md` registra las APIs reales y los archivos generados. El workflow E2E produce además un artefacto runtime. Así, teoría, práctica, fuente Java y evidencia de ejecución describen el mismo mecanismo.

Profundización del bloque 1 — Cuándo usar la vía simple y cuándo el exportador avanzado

La existencia de dos vías de exportación no es redundante. `JasperExportManager` es una fachada de conveniencia: recibe un `JasperPrint` y resuelve una salida PDF con valores por defecto. Resulta adecuada en utilidades pequeñas, prototipos o procesos

en los que la única decisión es la ruta de destino. En una aplicación empresarial, sin embargo, aparecen requisitos que no pertenecen al diseño visual del JRXML: título documental, autor, compresión, cifrado, contraseña, permisos o políticas distintas para copias internas y externas. Esos requisitos justifican `JRPdfExporter`.

La diferencia arquitectónica puede leerse como una progresión de responsabilidades. El JRXML define qué contiene el informe y cómo se presenta. `JasperFillManager` combina diseño, parámetros y datos y produce el `JasperPrint`. El exportador decide cómo serializar ese documento a un formato de distribución. Si una propiedad sólo afecta al fichero PDF y no al significado del informe, debe permanecer en la capa de exportación. Esta separación evita introducir en la plantilla decisiones que dependen del canal de entrega.

También cambia el tipo de destino. Una aplicación de escritorio puede escribir directamente en un archivo; un servicio web puede exportar a un `OutputStream` asociado a la respuesta HTTP; otro proceso puede necesitar un `byte[]` para firmarlo, cifrarlo externamente o almacenarlo en una base documental. La práctica usa archivos porque son fáciles de inspeccionar y conservar como evidencia, pero el modelo conceptual es el mismo.

En `EditorialReports` se mantiene un único llenado y se obtienen dos PDFs a partir del mismo `JasperPrint`: uno normal con metadatos verificables y otro protegido. Esta decisión demuestra que el documento de negocio no cambia por variar la política de distribución. Si ambos PDFs mostraran datos distintos, el problema estaría antes del exportador y no en la configuración PDF.

Profundización del bloque 2 — Metadatos como contrato documental

Los metadatos PDF tienen valor operativo además de descriptivo. En un repositorio documental permiten clasificar, buscar y presentar un archivo sin analizar visualmente todas sus páginas. El título debería describir el documento y no la ruta física del fichero; el autor identifica al área responsable; el asunto resume el propósito; las palabras clave facilitan indexación; el creador identifica la herramienta que produjo el artefacto.

Conviene distinguir estos metadatos de los textos impresos en las páginas. Cambiar `setMetadataTitle` no modifica el título visual del informe, porque ese título ya fue renderizado dentro del `JasperPrint`. Del mismo modo, cambiar el texto del título en el JRXML no obliga a que las propiedades del PDF se actualicen automáticamente. En sistemas reales ambas capas pueden mantenerse coordinadas mediante parámetros o una clase de configuración común, pero conceptualmente siguen siendo independientes.

El checkpoint prueba esa independencia. El documento conserva seis páginas y el mismo diseño de M5, mientras `pdfinfo` encuentra el título, autor y creador configurados. Esta prueba es más fuerte que buscar las cadenas dentro del Java: confirma que la librería las escribió efectivamente en la estructura PDF.

La API especifica `setMetadataTitle`, `setMetadataAuthor` y métodos equivalentes evita una ambigüedad frecuente al consultar ejemplos de versiones distintas. La fuente original utilizaba métodos que no pertenecen a `SimplePdfExporterConfiguration` en 6.20.0. En el curso se conserva la intención, pero la evidencia compilada tiene prioridad sobre la sintaxis histórica o aproximada.

Profundización del bloque 3 — Modelo de seguridad y límites de los permisos

Un PDF protegido combina dos conceptos: cifrado para controlar el acceso y permisos para indicar qué operaciones puede realizar un lector autorizado. La contraseña de usuario se relaciona con la apertura del documento. La contraseña de propietario protege la capacidad de modificar restricciones. Los permisos especifican acciones admitidas, como imprimir, copiar o utilizar lectores de pantalla.

No debe confundirse esta protección con un sistema completo de gestión de derechos digitales. Una contraseña PDF protege el documento dentro de las capacidades del estándar y del lector que lo interpreta, pero no sustituye controles de acceso de servidor, auditoría de descargas, expiración o clasificación de información. En una aplicación empresarial la seguridad suele comenzar antes: autorización para solicitar el informe, transporte HTTPS, almacenamiento protegido y políticas de retención.

El curso utiliza una contraseña conocida porque el objetivo es demostrar de forma automatizable que la protección existe. El workflow abre el fichero con `pdfinfo -upw editorial2026`. Si el archivo se hubiese generado sin cifrado, con otra contraseña o estuviera corrupto, la prueba no cumpliría el contrato.

También es importante la accesibilidad. El hint `SCREENREADERS` evita enseñar una política de seguridad que bloquee indiscriminadamente tecnologías de asistencia. El reto profesional consiste en equilibrar restricciones con el uso legítimo del documento. Los permisos deben responder al caso de negocio y no copiarse de forma automática entre todos los informes.

Profundización del bloque 4 — Reutilización, memoria y tratamiento de errores

Reutilizar un `JasperPrint` tiene dos ventajas. La primera es coherencia: todos los formatos proceden exactamente de los mismos datos, parámetros y evaluación de expresiones. La segunda es coste: una consulta, un cálculo de grupos, subreportes, gráficos y crosstabs puede ser sensiblemente más caro que serializar el resultado varias veces. M6 aprovecha el trabajo realizado durante el fill y desplaza la variación al último tramo del pipeline.

Esa reutilización también tiene implicaciones de memoria. `JasperPrint` contiene todas las páginas ya resueltas. En informes pequeños es natural mantenerlo en memoria mientras se generan varias salidas. En informes muy grandes habría que estudiar virtualización, procesamiento por lotes o políticas de generación distintas. El curso no introduce esa complejidad porque el catálogo es pequeño, pero conviene reconocer que “llenar una vez” no significa que el coste de memoria sea siempre irrelevante.

El bloque `try/catch` y `System.exit(1)` forman parte del contrato de ejecución. Una excepción de exportación no debe convertirse en un mensaje de consola seguido de código de salida cero, porque CI interpretaría falsamente que el trabajo terminó bien. Del mismo modo, crear previamente output evita confundir un fallo de directorio inexistente con un problema de JasperReports.

La secuencia correcta es deliberada: compilar subreporte, compilar maestro, construir parámetros, abrir conexión, llenar, exportar y validar. Cada fase deja una evidencia distinta y permite localizar con precisión dónde se rompe el proceso.

Profundización del bloque 5 — Validación de un PDF más allá de “existe el archivo”

Una validación profesional no debería aceptar un archivo únicamente porque `File.exists()` sea verdadero. Un proceso podría crear un fichero vacío, truncado o con una extensión equivocada. Por eso el E2E comprueba primero la firma %PDF- y después utiliza `pdfinfo`, una herramienta independiente del código Java que generó el archivo.

La firma confirma que el contenido tiene estructura PDF. `pdfinfo` obliga a que el documento sea parseable y, además, permite observar páginas y metadatos. Para la versión protegida se suministra la contraseña de usuario documentada. Esta combinación cubre varias clases de error sin necesidad de inspección manual.

En un sistema productivo podrían añadirse controles de tamaño mínimo, número de páginas esperado, presencia de texto, firma digital, PDF/A o comparación visual de páginas críticas. El curso mantiene el alcance en los requisitos del punto 6.1 y añade los invariantes heredados del proyecto para demostrar que la exportación no ha alterado el informe.

La trazabilidad física complementa esas pruebas. `audit_m6_traceability.py` compara los checkpoints y no permite que el módulo de exportación cambie silenciosamente el JRXML o el JRTX ya cerrado en M5. De este modo, cuando aparece una diferencia en el PDF de M6, se sabe que procede de la exportación o de la configuración y no de un rediseño oculto.

Caso profesional — Diseñar un servicio PDF verificable

Imaginemos que `EditorialReports` deja de ejecutarse manualmente y pasa a formar parte de una aplicación interna. El usuario solicita “Informe de ventas”, la aplicación llena el `JasperReport` y debe ofrecer una copia para lectura normal y otra protegida para intercambio externo. El diseño estudiado en 6.1 sigue siendo válido porque el generador puede separar el `JasperPrint` de la política de salida.

La aplicación podría recibir el período y el usuario como parámetros, construir el documento una sola vez y seleccionar un perfil PDF. El perfil interno conservaría metadatos y compresión sin contraseña; el externo activaría cifrado y permisos restringidos. El código de negocio no necesitaría duplicar consultas ni mantener dos JRXML. Esta reutilización reduce el riesgo de que dos “versiones del mismo informe” terminen mostrando cifras distintas.

La aceptación del servicio debería comprobar varias capas. Primero, que el llenado produce el número esperado de páginas y los invariantes de datos. Segundo, que el fichero tiene estructura PDF y puede ser leído por una herramienta independiente. Tercero, que los metadatos identifican correctamente el documento. Cuarto, que las restricciones de la copia protegida se aplican y la contraseña documentada permite abrirla. Finalmente, una revisión visual distribuida confirma que el exportador no ha introducido problemas de fuentes o representación.

Este ejemplo permite distinguir claramente errores de dominio, de diseño y de exportación. Si el total es incorrecto en todos los formatos, hay que investigar consulta, parámetros o expresiones. Si sólo falla el PDF protegido, la investigación debe centrarse en `JRPdfExporter` y su configuración. Si el archivo es correcto pero la aplicación no puede entregarlo, el problema está en la capa de publicación. Esa capacidad de aislar responsabilidades es uno de los objetivos profesionales del módulo.

Punto 6.2 — Exportación a Excel

Objetivos de aprendizaje

- Comprender las diferencias entre los formatos XLS y XLSX.
- Utilizar el exportador `JRXLsxExporter` para generar archivos Excel modernos.
- Configurar las propiedades del exportador mediante `SimpleXlsxExporterConfiguration`.
- Ajustar el nombre de la hoja, el ancho de columnas y las celdas combinadas.
- Aplicar formato a las celdas exportadas.
- Documentar la exportación a Excel del proyecto `EditorialReports`.

Bloque 1 — XLS frente a XLSX y dependencia de Apache POI

El segundo punto añade Excel al mismo `JasperPrint` que ya se exporta a PDF. XLS es el formato binario histórico de Excel; XLSX es el formato OOXML moderno, empaquetado como ZIP y compuesto por documentos XML. `EditorialReports` utiliza `JRXLsxExporter` porque el objetivo es producir `output/informe_ventas.xlsx`.

`JasperReports 6.20.0` declara Apache POI como dependencia opcional. Por eso un proyecto Maven que sólo había generado PDF puede compilar `JasperReports` y, sin embargo, fallar en tiempo de ejecución al usar XLSX si POI no está en el classpath. El checkpoint 6.2 añade explícitamente `poi` y `poi-ooxml 5.1.0` al `pom.xml`.

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
```

XML

```
<version>5.1.0</version>
</dependency>
```

El E2E comprueba que Maven resuelve las dependencias, que el archivo empieza por la firma PK propia de ZIP y que el paquete OOXML no contiene entradas corruptas. Así la dependencia no queda como una nota teórica: forma parte del contrato ejecutable.

Bloque 2 — Dos niveles de configuración XLSX

La fuente original concentraba nombre de hoja, cuadrícula, bloqueo y paleta en `SimpleXlsxExporterConfiguration`. En la API real esas opciones se dividen. `SimpleXlsxReportConfiguration` controla cómo un `JasperPrint` se transforma en hojas y celdas; `SimpleXlsxExporterConfiguration` contiene opciones del libro/exportador.

```
SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();
informe.setSheetNames(new String[]{"Ventas"});
informe.setShowGridLines(Boolean.FALSE);
informe.setCellLocked(Boolean.FALSE);
informe.setCellHidden(Boolean.FALSE);
informe.setDetectCellType(Boolean.TRUE);
informe.setOnePagePerSheet(Boolean.FALSE);

SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();
libro.setCreateCustomPalette(Boolean.TRUE);
```

JAVA

Separar ambas configuraciones evita llamar métodos sobre una clase que no los define. `setDetectCellType` intenta conservar valores numéricos como tipos Excel en lugar de convertir todo a texto. `setOnePagePerSheet` (FALSE) evita crear una hoja por cada página del `JasperPrint`.

Bloque 3 — Flujo de JRXlsxExporter

`JRXlsxExporter` recibe el mismo documento ya utilizado por PDF. Se aplican las dos configuraciones, se establece `SimpleExporterInput` y la salida binaria con `SimpleOutputStreamExporterOutput`.

```
JRXlsxExporter exportador = new JRXlsxExporter();
exportador.setConfiguration(informe);
exportador.setConfiguration(libro);
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(rutaXlsx));
exportador.exportReport();
```

JAVA

JasperReports convierte coordenadas y elementos gráficos a una rejilla de celdas. Esa traducción nunca es idéntica a un PDF porque Excel trabaja con filas y columnas, pero conserva información útil de texto, números y estilos. Cuanto más tabular sea el informe, más natural será el resultado.

El diseño JRXML no cambia. La auditoría compara byte a byte `informe_ventas.jrxml` y `EditorialStyles.jrtx` con M5/5.6. El objetivo del punto es el exportador, no rediseñar el informe para Excel.

Bloque 4 — Nombre de hoja, cuadrícula, tipos y edición

El nombre Ventas se aplica mediante `setSheetNames`. El workflow abre internamente `xl/workbook.xml` del XLSX y exige que exista una hoja con ese nombre. Esto convierte un requisito de la práctica en una prueba automática.

`setShowGridLines` (FALSE) elimina las líneas de cuadrícula predeterminadas. `setCellLocked` (FALSE) y `setCellHidden` (FALSE) dejan las celdas sin protección adicional. `setCreateCustomPalette` (TRUE) intenta conservar los colores del informe dentro de las limitaciones del formato/exportador.

```
JasperPrint (6 páginas)
  |
  v
JRXlsxExporter
  |
  +-- ReportConfiguration
  |   | sheet = Ventas
  |   | gridlines = false
  |   | detectCellType = true
  |   +-- ExporterConfiguration
  |       | custom palette = true
  |       +--
  |
  v
informe_ventas.xlsx
```

TEXT

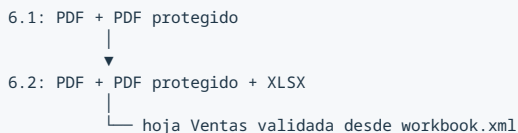
La práctica no afirma que cada píxel del PDF tenga una equivalencia exacta en Excel. Enseña qué propiedades controla el exportador y cómo verificar el resultado estructural.

Bloque 5 — Validación OOXML y continuidad acumulativa

Un fichero con extensión `.xlsx` no es suficiente evidencia. El E2E comprueba tamaño mayor que cero, firma ZIP, integridad de todas las entradas y presencia de la hoja Ventas. A la vez, vuelve a ejecutar los cinco generadores heredados y comprueba los invariantes SQLite.

El checkpoint 6.2 es físicamente 6.1 más: dos dependencias Maven, la lógica XLSX en `GeneradorInformeVentas.java` y `EXPORTACION_EXCEL.md`. No elimina la exportación PDF ni el PDF protegido.

TEXT



Esta acumulación permite que los puntos posteriores reutilicen el mismo XLSX sin duplicar el llenado. La documentación distingue además la configuración por informe y la configuración por exportador para impedir que reaparezca el error técnico de la fuente original.

Profundización del bloque 1 — El cambio de un lienzo paginado a una rejilla de celdas

PDF y XLSX representan el mismo informe con modelos muy diferentes. PDF conserva una geometría de página: cada elemento tiene una posición y un tamaño concretos. XLSX trabaja con una rejilla de filas y columnas. El exportador debe traducir posiciones del `JasperPrint` a límites de celdas, decidir cuándo combinar celdas y mantener, en la medida de lo posible, tipos, bordes, alineaciones y colores.

Esta traducción explica por qué un informe que se ve perfecto en PDF puede generar un Excel poco cómodo para analizar. Si muchos elementos empiezan o terminan en coordenadas ligeramente distintas, el exportador necesita crear más columnas para respetar la geometría. Por eso los informes destinados a análisis tabular suelen diseñarse con una alineación rigurosa. En M6 no se rediseña `informe_ventas.jrxml` porque el objetivo es estudiar el comportamiento del exportador sobre el diseño ya cerrado en M5.

XLSX es además un paquete OOXML. El archivo que el usuario percibe como un único libro es realmente un ZIP con documentos XML internos, relaciones, estilos y hojas. Esta propiedad permite al E2E validar la salida sin depender de Microsoft Excel: basta abrir el ZIP, comprobar su integridad y leer `xl/workbook.xml` para confirmar el nombre de hoja.

La dependencia POI tiene sentido en este contexto. JasperReports delega parte de la construcción del libro en Apache POI. Que esa dependencia sea opcional evita imponerla a proyectos que nunca exportan a Office, pero obliga a declararla cuando se utiliza XLSX. El curso convierte esa circunstancia en una decisión explícita del `pom.xml`.

Profundización del bloque 2 — ReportConfiguration y ExporterConfiguration como ámbitos diferentes

La separación entre `SimpleXlsxReportConfiguration` y `SimpleXlsxExporterConfiguration` expresa dos ámbitos de decisión. El primero describe cómo debe interpretarse un informe concreto al producir hojas: nombres, cuadrícula, detección de tipos, protección de celdas o relación entre páginas y hojas. El segundo contiene opciones propias del libro/exportador, como la paleta.

Esta distinción es importante cuando un exportador recibe varios `JasperPrint` o cuando una aplicación quiere reutilizar una política global con informes que necesitan ajustes diferentes. Mezclar ambos ámbitos en una sola clase haría más difícil decidir qué opción pertenece al documento y cuál pertenece al proceso de exportación.

En el checkpoint, `setSheetNames` se aplica al report configuration porque el nombre “Ventas” describe la hoja resultante del informe de ventas. `setCreateCustomPalette` se aplica al exporter configuration porque afecta a cómo el libro gestiona colores. Maven confirma que esa separación corresponde a la API 6.20.0; no se trata sólo de una recomendación de estilo.

Una consecuencia pedagógica es que no se debe memorizar una lista de métodos sin observar el tipo del objeto receptor. En Java, dos configuraciones con nombres parecidos pueden tener responsabilidades distintas. La forma fiable de trabajar es identificar primero qué aspecto se quiere controlar y después buscarlo en la interfaz de configuración adecuada.

Profundización del bloque 3 — Tipos de celda, patrones y valor útil para análisis

`setDetectCellType(Boolean.TRUE)` merece especial atención. Un Excel útil para el departamento comercial no debería convertir números en texto si puede conservarlos como valores numéricos. Una celda numérica puede sumarse, filtrarse y utilizarse en fórmulas; una cadena con apariencia de número obliga a transformaciones posteriores.

JasperReports parte del tipo Java y del patrón de presentación del elemento. El valor del field y el patrón no son lo mismo: el primero determina el dato lógico y el segundo cómo se muestra. El exportador intenta trasladar esa información a Excel. Un importe puede seguir siendo numérico mientras Excel aplica un formato visual; una fecha puede conservar un valor de fecha con un formato legible.

No todos los elementos de un informe paginado son igualmente útiles en una hoja de cálculo. Títulos, pies de página, líneas decorativas, subreportes o gráficos responden a necesidades de presentación. La exportación conserva todo lo posible porque la práctica trabaja con el mismo informe multiformato, pero en proyectos orientados a explotación de datos puede ser razonable mantener una plantilla específica o utilizar propiedades de exportación para omitir elementos.

El curso evita prometer identidad visual entre PDF y XLSX. El criterio correcto es doble: que el libro sea válido y que conserve de forma razonable la información y la organización del informe. La inspección humana de Excel complementa las comprobaciones estructurales automáticas cuando el aspecto final sea un requisito contractual.

Profundización del bloque 4 — El segundo XLSX del reto y la reutilización del exportador

El reto de 6.2 no se limita a cambiar el nombre de una hoja. Genera un segundo `JasperPrint` a partir de `catalogo.csv` mediante `JRCsvDataSource` y el informe de catálogo heredado. Después reutiliza el mismo método `exportarXlsx` con el nombre de hoja `Catálogo`.

Esta solución demuestra dos formas de reutilización. Por una parte, el método de exportación no conoce el origen de los datos: recibe un `JasperPrint`, una ruta y un nombre de hoja. Por otra, la aplicación puede producir libros distintos a partir de documentos distintos sin duplicar toda la configuración XLSX.

El `JRCsvDataSource` utiliza la primera fila como cabecera para que los nombres de columna del CSV se correspondan con los fields del informe. El bloque `finally` cierra el `datasource` incluso si el fill o la exportación fallan. Este detalle es importante porque una práctica de exportación también debe enseñar el ciclo de vida de los recursos.

El E2E abre ambos XLSX y exige las hojas `Ventas` y `Catálogo`. Con ello el reto queda probado en el mismo nivel que el ejercicio principal. No existe una “solución de papel” separada del código: la práctica documenta el mismo estado que el checkpoint ejecutable.

Profundización del bloque 5 — Qué validar en un archivo XLSX

La firma PK sólo indica que el fichero parece un ZIP; no demuestra que sea un libro Excel íntegro. Por eso el workflow recorre las entradas mediante `zipfile` y ejecuta `testzip()`. Después analiza `xl/workbook.xml` con un parser XML y comprueba los nombres de hoja.

En una validación más extensa podrían inspeccionarse también `xl/worksheets/sheetN.xml`, `styles.xml`, tipos de celda, fórmulas o valores concretos. Apache POI podría reabrir el libro y validar propiedades a un nivel semántico superior. Para el alcance del curso, la integridad del paquete y el contrato de las hojas proporcionan una evidencia robusta sin acoplar la prueba a una aplicación gráfica.

Es igualmente importante mantener la trazabilidad con el `pom.xml`. Si se eliminan `poi` o `poi-ooxml` y el proyecto deja de disponer de las clases necesarias en tiempo de ejecución, el E2E debe fallar. De este modo, dependencias, código y artefacto final forman una cadena única.

La comparación acumulativa 6.1→6.2 permite además atribuir el cambio: se añaden las dependencias, el código XLSX, la documentación y el segundo libro del reto, mientras el JRXML principal permanece intacto. Esta disciplina evita resolver un problema de exportación mediante modificaciones silenciosas del informe.

Caso profesional — Diseñar un informe que también sea útil en Excel

Cuando un informe se sabe desde el principio que tendrá salida PDF y XLSX, conviene alinear los elementos de detalle con una rejilla consistente. Dos campos que visualmente parecen alineados pero comienzan con una diferencia mínima pueden obligar al exportador a crear columnas adicionales. El resultado puede ser correcto pero incómodo para filtrar o copiar.

Eso no significa que el diseño deba renunciar a la presentación. Significa que la exportabilidad se convierte en un requisito de arquitectura del informe. Los encabezados principales pueden conservarse para PDF y la zona tabular puede mantenerse rigurosamente alineada. JasperReports dispone además de propiedades específicas para controlar determinados comportamientos de exportación cuando un proyecto necesita una optimización mayor.

El segundo XLSX del reto demuestra otro patrón habitual: no todo libro Excel tiene que proceder del mismo informe. El informe de ventas conserva componentes complejos, mientras el catálogo parte de una fuente CSV y posee una estructura más directamente tabular. Un servicio real puede elegir diferentes JasperReports según el objetivo del formato y aun así reutilizar la misma política de exportación.

La validación debería comprobar no sólo que Excel abre el archivo. Para un contrato empresarial pueden verificarse nombres de hoja, número mínimo de filas, presencia de columnas clave, tipos numéricos, formatos de fecha y ausencia de hojas inesperadas. El curso valida el paquete OOXML y los nombres de hoja porque son requisitos deterministas del ejercicio.

Este enfoque ayuda a separar dos preguntas: “¿el archivo XLSX es técnicamente válido?” y “¿es útil para el usuario que lo va a analizar?”. La primera se automatiza; la segunda combina requisitos de negocio e inspección funcional. Un buen proyecto necesita ambas.

Punto 6.3 — Exportación a HTML

Objetivos de aprendizaje

- Comprender el papel del exportador HTML y sus limitaciones respecto a PDF.
- Configurar el exportador `JRHtmlExporter` con `SimpleHtmlExporterConfiguration`.
- Exportar las imágenes del informe a un directorio y referenciarlas desde el HTML.

- Añadir cabecera, pie y separador de páginas al archivo HTML.
- Integrar el HTML generado con una hoja de estilos CSS externa.
- Documentar la exportación a HTML del proyecto EditorialReports.

Bloque 1 — HTML como exportación navegable del JasperPrint

HTML representa el informe mediante marcado que un navegador puede interpretar. A diferencia del PDF, el resultado puede depender de recursos externos —imágenes y CSS—, por lo que una validación correcta debe considerar el archivo principal y esos recursos. JasperReports 6.20.0 utiliza `HtmlExporter`; el nombre `JRHtmlExporter` del material original no es la clase empleada por el checkpoint compilado.

JAVA

```
HtmlExporter exportador = new HtmlExporter();
exportador.setExporterInput(new SimpleExporterInput(documento));
```

El mismo `JasperPrint` de seis páginas alimenta el exportador. No se ejecuta de nuevo `SQLite` y no se modifica `informe_ventas.jrxml`. El checkpoint crea `output/informe_ventas.html`, `output/images/` y `output/styles/editorial.css`.

Bloque 2 — SimpleHtmlExporterConfiguration: cabecera, pie y páginas

`SimpleHtmlExporterConfiguration` controla elementos HTML generales. El checkpoint define una cabecera completa con UTF-8, título y enlace a CSS; define también el cierre del documento y un separador entre páginas del `JasperPrint`.

JAVA

```
SimpleHtmlExporterConfiguration configuracion = new SimpleHtmlExporterConfiguration();
configuracion.setHtmlHeader("<html><head><meta charset='UTF-8'>"
    + "<title>Informe de Ventas - EditorialReports</title>"
    + "<link rel='stylesheet' href='styles/editorial.css'>"
    + "</head><body>");
configuracion.setHtmlFooter("</body></html>");
configuracion.setBetweenPagesHtml("<hr class='salto-pagina' />");
```

La codificación se declara en el HTML y también en el objeto de salida. El enlace CSS es relativo al HTML final, por eso el archivo se copia a `output/styles`. El separador permite aplicar un estilo específico entre páginas sin introducir contenido en el `JRXML`.

Bloque 3 — SimpleHtmlExporterOutput y recursos de imagen

La fuente original asignaba rutas de imágenes mediante métodos de `SimpleHtmlExporterConfiguration`. En el modelo real, el tratamiento de recursos corresponde al output. El checkpoint crea `SimpleHtmlExporterOutput` y le asigna un `FileHtmlResourceHandler`.

JAVA

```
SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(rutaHtml, "UTF-8");
salida.setImageHandler(
    new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));
);
exportador.setExporterOutput(salida);
```

El handler conoce dónde escribir físicamente los recursos y qué URI debe aparecer en el HTML. Así el navegador puede resolver `images/...` desde el archivo ubicado en `output`.

Aunque una ejecución concreta pueda necesitar cero o varias imágenes externas, la carpeta y la estrategia quedan preparadas. El E2E exige que el directorio exista y que el HTML sea coherente con la estructura documentada.

Bloque 4 — CSS externo y separación de presentación web

`editorial.css` no sustituye los estilos JasperReports. Los estilos `JRTX` siguen controlando el `JasperPrint`; el CSS añade presentación al contenedor HTML que rodea el contenido exportado.

CSS

```
body {
    margin: 24px;
    background: #ffffff;
    color: #173f6b;
    font-family: "DejaVu Sans", Arial, sans-serif;
}
.salto-pagina {
    border-top: 1px solid #d6eaf8;
}
```

Esta separación evita incrustar grandes cantidades de CSS en Java y hace visible en el árbol del proyecto el recurso que debe publicarse junto al HTML. La práctica visual enseña a crear el CSS, copiarlo y validar el enlace.

El E2E comprueba que el HTML contiene meta charset, título, referencia styles/editorial.css y etiqueta de cierre; también exige que el CSS exista en la salida.

Bloque 5 — Validación funcional del HTML

La validación se realiza en varias capas. Maven verifica que las clases y métodos existen. La ejecución verifica que el exportador serializa el JasperPrint real. Las aserciones estructurales comprueban las piezas mínimas del documento y sus recursos.

TEXT



El resultado se acumula sobre 6.2: PDF, PDF protegido y XLSX siguen generándose en la misma ejecución. Esta continuidad está verificada por el workflow, no sólo descrita en el texto.

Profundización del bloque 1 — HTML no es “un PDF dentro del navegador”

Aunque ambos formatos parten del mismo JasperPrint, HTML y PDF tienen objetivos distintos. PDF fija páginas y está pensado para impresión o distribución cerrada. HTML vive dentro de un navegador, utiliza un modelo de caja, puede cargar recursos mediante URI y participa en una página web más amplia. El exportador de JasperReports intenta conservar la disposición del informe, pero el resultado sigue sujeto a las reglas del navegador.

Por esa razón, la práctica mantiene una carpeta de publicación coherente: el HTML principal, la subcarpeta de imágenes y la subcarpeta de estilos. Si una URI relativa no coincide con la estructura física, el documento puede abrirse sin estilos o con imágenes rotas aunque el Java haya terminado sin excepción.

HtmlExporter es la clase utilizada en 6.20.0. La sustitución del nombre histórico del material original no cambia el concepto: existe un exportador especializado que recibe un JasperPrint y produce marcado. Lo importante es que la clase documentada sea la que realmente compila en el proyecto.

El E2E no necesita un navegador para confirmar los contratos básicos. Puede abrir el HTML como texto, verificar charset, título, enlace CSS, enlace PDF y etiqueta de cierre. La comprobación visual en navegador sigue siendo útil para diseño, pero la estructura mínima ya queda automatizada.

Profundización del bloque 2 — Cabecera y pie como frontera entre JasperReports y la aplicación web

setHtmlHeader y setHtmlFooter permiten envolver el contenido exportado con la estructura que necesita la aplicación. Esto evita modificar el JRXML sólo para insertar elementos que pertenecen al canal web, como una etiqueta meta, una hoja CSS o un enlace de navegación.

En el checkpoint la cabecera declara UTF-8, un título y styles/editorial.css. El pie cierra body y html. setBetweenPagesHtml inserta un separador entre las páginas lógicas del JasperPrint. El documento sigue recordando que procede de seis páginas, pero el navegador puede presentarlas en un flujo continuo.

Esta frontera es útil cuando el mismo informe se publica en contextos distintos. Una intranet podría envolver el contenido con su navegación corporativa; una aplicación pública podría añadir cabeceras de accesibilidad o una hoja de estilos diferente; un correo HTML necesitaría restricciones adicionales. El JRXML seguiría concentrado en el contenido del informe.

También conviene evitar introducir datos no confiables directamente en la cabecera HTML sin escapar. En esta práctica las cadenas son constantes controladas por el proyecto, por lo que no existe una superficie de inyección procedente del usuario. Si el título o enlaces fueran parámetros externos habría que tratarlos como cualquier otro contenido web.

Profundización del bloque 3 — ResourceHandler y resolución de rutas

Un HTML puede referirse a recursos con URI relativas aunque esos recursos se escriban en una ruta física distinta.

FileHtmlResourceHandler conecta ambos mundos: recibe el directorio donde JasperReports debe guardar los recursos y el patrón de URI que se insertará en el marcado.

El directorio output/images pertenece al sistema de archivos. La URI images/{0} pertenece al documento HTML. Como informe_ventas.html vive en output, ambas referencias coinciden: el navegador resolverá images/. . . dentro de la subcarpeta situada junto al HTML.

Este principio es fundamental al desplegar en servidores. Si el HTML se mueve pero sus imágenes no, las URI dejan de resolver. Si se publica bajo un prefijo web distinto, quizá convenga un handler que genere URL absolutas o que almacene recursos en otro servicio. El checkpoint utiliza rutas relativas porque el artifact debe poder abrirse de forma autónoma.

La carpeta puede quedar vacía si el informe no requiere recursos rasterizados externos en una ejecución concreta; eso no invalida la configuración. Lo que se valida es que el mecanismo y el directorio están disponibles para cuando el exportador los necesite.

Profundización del bloque 4 — CSS externo, JRTX y responsabilidad de cada capa

El CSS de 6.3 y `EditorialStyles.jrtx` no son alternativas. JRTX participa antes, durante el fill: define estilos de elementos que terminan formando parte del `JasperPrint`. CSS participa después, cuando ese `JasperPrint` ya se ha convertido a HTML y la aplicación quiere estilizar el contenedor web.

El checkpoint usa CSS para el body, la representación de páginas y el separador. No intenta reconstruir manualmente cada estilo JasperReports mediante selectores CSS. Esa estrategia reduciría la trazabilidad y obligaría a mantener dos definiciones visuales completas.

La copia de `resources/styles/editorial.css` a `output/styles/editorial.css` forma parte del proceso de publicación. El código utiliza `StandardCopyOption.REPLACE_EXISTING` para evitar que un archivo viejo permanezca en la salida después de cambiar el recurso fuente.

En entornos de producción podría emplearse un pipeline de frontend que versionara o minificara el CSS. Para el curso, mantener el recurso dentro del proyecto y copiarlo de forma determinista facilita reproducir el resultado y empaquetarlo junto al artifact.

Profundización del bloque 5 — El enlace al PDF y la publicación multicanal

El reto añade al HTML un enlace `Descargar PDF` que apunta a `informe_ventas.pdf`. La decisión es coherente con el pipeline: el PDF normal se genera antes de exportar HTML y ambos archivos terminan en la misma carpeta `output`. Por ello basta una URI relativa.

Este pequeño requisito demuestra una idea mayor: los formatos no tienen por qué vivir aislados. La versión HTML puede actuar como punto de navegación hacia la versión imprimible; una aplicación web podría ofrecer además XLSX o CSV según los permisos del usuario. El módulo empieza así a parecerse a un servicio real de publicación de informes.

El E2E busca tanto el texto del enlace como su `href`. Si el generador centralizado de 6.5 olvidara conservar la cabecera personalizada, esa prueba detectaría la regresión. De hecho, la revalidación final se diseñó precisamente para asegurar que la refactorización no eliminara el reto HTML.

Una validación manual posterior debería abrir el HTML desde su carpeta de salida, comprobar estilos, saltos y enlace. Pero la prueba automática ya garantiza que la estructura necesaria existe y que el PDF destino se genera en la misma ejecución.

Un último aspecto es la portabilidad del conjunto publicado. Si sólo se copia `informe_ventas.html` y se olvidan `styles` o `images`, el archivo deja de ser autocontenido desde el punto de vista operativo. Por eso el artifact del checkpoint conserva la estructura de directorios y no trata el HTML como una única salida aislada. En un despliegue web real esa misma idea puede traducirse a un paquete estático, un directorio servido por un reverse proxy o recursos almacenados en una CDN.

La codificación también debe mantenerse coherente de extremo a extremo. La cabecera declara UTF-8 y `SimpleHtmlExporterOutput` escribe UTF-8. Si ambas decisiones divergieran, caracteres como tildes, eñes o el símbolo del euro podrían interpretarse de forma distinta por el navegador. La práctica demuestra que configuración documental, writer y recursos forman un único contrato de publicación.

Caso profesional — Publicar el informe en una intranet

Supongamos que el departamento comercial quiere consultar el informe desde una intranet y descargar la copia PDF si necesita imprimirla. El flujo de 6.3 encaja directamente: se genera el PDF, después el HTML, se publica el CSS y el enlace relativo conecta ambos formatos.

Para que el despliegue sea fiable, el directorio completo debe tratarse como una unidad. El servidor web tendría que publicar `informe_ventas.html`, `informe_ventas.pdf`, `styles/editorial.css` y cualquier recurso de `images` manteniendo sus rutas relativas. Si el equipo de operaciones mueve sólo el HTML a otra carpeta, el Java habrá terminado correctamente pero la experiencia web quedará rota.

En una aplicación real sería razonable añadir pruebas HTTP después del despliegue: solicitar el HTML, comprobar código 200, recuperar el CSS, seguir el enlace PDF y verificar el tipo MIME. Esas pruebas pertenecen a la capa de despliegue, mientras el E2E actual se concentra en el paquete de archivos producido por JasperReports.

El HTML también plantea decisiones de accesibilidad. La estructura generada por un motor orientado a informes puede no equivaler a una página semántica diseñada manualmente. Si la accesibilidad web es un requisito central, conviene inspeccionar encabezados, orden de lectura, alternativas de imagen y navegación con teclado. La posibilidad de ofrecer simultáneamente PDF no elimina esa obligación.

Así, 6.3 no enseña simplemente a “guardar como HTML”. Enseña que el formato web introduce recursos, rutas y un entorno de ejecución diferente. El exportador es sólo una pieza del proceso de publicación.

Punto 6.4 — Exportación a CSV y otros formatos

Objetivos de aprendizaje

- Comprender las características del formato CSV y sus limitaciones.
- Utilizar el exportador `JRCsvExporter` con `SimpleCsvExporterConfiguration`.
- Configurar el separador de campos y la codificación del archivo CSV.
- Exportar a formato XML con `JRXmlExporter` y a RTF con `JRRtfExporter`.
- Combinar varios exportadores en una misma ejecución del programa.
- Documentar la exportación a CSV, XML y RTF del proyecto `EditorialReports`.

Bloque 1 — CSV: exportación orientada a datos

CSV sacrifica la geometría de página para producir texto delimitado. `JRCsvExporter` recorre el contenido textual exportable del `JasperPrint` y genera registros. En `EditorialReports` se utiliza punto y coma como delimitador de campos y salto de línea como delimitador de registros.

JAVA

```
SimpleCsvExporterConfiguration configuracion = new SimpleCsvExporterConfiguration();
configuracion.setFieldDelimiter(";");
configuracion.setRecordDelimiter("\n");
configuracion.setWriteBOM(Boolean.TRUE);
```

La fuente original proponía `setEncoding("UTF-8")` sobre `SimpleCsvExporterConfiguration`. En el checkpoint real, la codificación pertenece al `ExporterOutput`:

JAVA

```
exportador.setExporterOutput(new SimpleWriterExporterOutput(rutaCsv, "UTF-8"));
```

El BOM facilita que aplicaciones de escritorio detecten UTF-8. El E2E comprueba los bytes EF BB BF, la presencia de punto y coma y que exista más de una línea. Por tanto, la prueba valida el fichero producido, no sólo la llamada Java.

Bloque 2 — XML: representación estructural del documento

`JRXmlExporter` serializa el `JasperPrint` a XML. No produce el XML de negocio original ni ejecuta una consulta diferente: representa el documento ya llenado. `SimpleXmlExporterOutput` permite definir archivo y codificación.

JAVA

```
JRXmlExporter exportador = new JRXmlExporter();
SimpleXmlExporterOutput salida = new SimpleXmlExporterOutput(rutaXml, "UTF-8");
salida.setEmbeddingImages(Boolean.TRUE);
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(salida);
exportador.exportReport();
```

Con `setEmbeddingImages(Boolean.TRUE)` los recursos gráficos pueden quedar embebidos según la representación del exportador, evitando referencias externas. La prueba automática exige que el fichero comience con una declaración XML.

Bloque 3 — RTF: salida para procesadores de texto

`JRRtfExporter` produce Rich Text Format. La salida es textual y el checkpoint utiliza `SimpleWriterExporterOutput(rutaRtf, "UTF-8")`.

JAVA

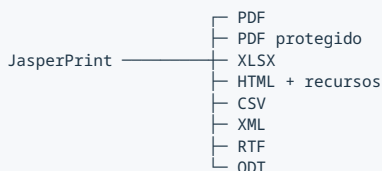
```
JRRtfExporter exportador = new JRRtfExporter();
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleWriterExporterOutput(rutaRtf, "UTF-8"));
exportador.exportReport();
```

La fuente original trasladaba la codificación a una configuración RTF. La implementación corregida la aplica al writer. El E2E verifica la cabecera RTF del archivo. Como reto acumulativo, el punto añade `net.sf.jasperreports.engine.export.oasis.JR0dtExporter` para producir `output/informe_ventas.odt` a partir del mismo `JasperPrint`.

Bloque 4 — Un JasperPrint, varios formatos

Al llegar a 6.4 una sola ejecución de `GeneradorInformeVentas` produce PDF, PDF protegido, XLSX, HTML, CSV, XML, RTF y ODT. Todos parten del mismo objeto documento.

TEXT



Esta arquitectura es más eficiente que volver a llenar el informe para cada formato. Los parámetros, consulta y totales son idénticos para todas las salidas. Si una exportación lanza excepción, el catch final registra la traza y `System.exit(1)` hace visible el fallo en CI.

Bloque 5 — Contratos de archivo y E2E multiformato

Cada formato necesita una evidencia distinta. El workflow aplica contratos estructurales: PDF con firma PDF, XLSX como ZIP OOXML y hojas Ventas/Catálogo, HTML con etiquetas y CSS, CSV con BOM/delimitador, XML con declaración XML, RTF con su firma y ODT como paquete OpenDocument válido.

TEXT

Formato	Evidencia mínima
PDF	firma PDF + pdfinfo
XLSX	PK + ZIP íntegro + workbook.xml
HTML	charset + título + CSS + cierre
CSV	BOM UTF-8 + ; + registros
XML	declaración XML
RTF	cabecera RTF
ODT	ZIP íntegro + mimetype OpenDocument Text

A la vez, el workflow comprueba SQLite y las seis páginas de `informe_ventas`. `EXPORTACION_OTROS.md` documenta las decisiones corregidas y el JRXML/JRTX continúan byte a byte iguales a M5/5.6.

Profundización del bloque 1 — CSV como formato de intercambio y sus pérdidas deliberadas

CSV no intenta conservar páginas, fuentes, bordes o gráficos. Su objetivo es producir un flujo tabular que pueda ser leído por hojas de cálculo, procesos ETL, scripts o sistemas externos. Esta diferencia de propósito explica por qué una exportación CSV no debe juzgarse con el mismo criterio visual que el PDF.

El delimitador de campos depende del ecosistema. En muchos entornos europeos se utiliza punto y coma porque la coma se reserva para decimales. El checkpoint fija ; y un salto de línea como separador de registros. La elección debe ser conocida por el consumidor; un CSV no lleva un esquema capaz de describir automáticamente todos estos detalles.

El BOM UTF-8 es una decisión de interoperabilidad. UTF-8 no necesita BOM desde el punto de vista del estándar, pero algunas aplicaciones de escritorio lo utilizan para detectar la codificación. `setWriteBOM(Boolean.TRUE)` resuelve esa necesidad concreta sin desplazar la codificación al objeto equivocado: la escritura UTF-8 se define en `SimpleWriterExporterOutput`.

La exportación parte del `JasperPrint`, no directamente de una consulta SQL. Por ello el contenido CSV representa aquello que JasperReports considera exportable del documento. Cuando un proyecto necesita un fichero de datos puro y estable como interfaz de integración, puede ser más apropiado generar ese contrato directamente desde el dominio. En este curso interesa comparar formatos partiendo del mismo documento.

Profundización del bloque 2 — XML de JasperPrint frente a XML de negocio

El XML generado por `JRXmlExporter` describe el documento resultante de JasperReports. No debe confundirse con `distribucion.xml` ni con un XML diseñado como contrato de negocio. Un XML de negocio modela conceptos como libros, autores o entregas; el XML de exportación conserva la estructura necesaria para representar el informe.

Esta distinción evita una decisión arquitectónica equivocada: utilizar el XML exportado como si fuera una API estable para otros sistemas. Puede ser útil para inspección, archivado técnico o procesos que conozcan el modelo JasperReports, pero un servicio externo normalmente debería consumir un esquema propio del dominio.

`SimpleXmlExporterOutput` define ruta y codificación. `setEmbeddingImages(Boolean.TRUE)` indica que los recursos gráficos deben viajar embebidos cuando el exportador lo permita, simplificando el transporte del resultado. El E2E comprueba que el documento sea un XML real al exigir la declaración inicial y un archivo no vacío.

En una validación más estricta podría parsearse completamente el XML, contar páginas o buscar elementos concretos. El curso combina la prueba estructural con el control de que el `JasperPrint` origen tiene seis páginas y conserva los invariantes de negocio.

Profundización del bloque 3 — RTF, procesadores de texto y fidelidad esperable

RTF está orientado a procesadores de texto y posee un modelo de maquetación diferente al de PDF. JasperReports intenta trasladar textos, estilos y estructura, pero no debe prometerse una reproducción exacta de cada coordenada. La utilidad principal es ofrecer un documento que el destinatario pueda abrir y editar con herramientas de oficina.

`JRRTfExporter` utiliza una salida de escritor. La codificación se configura en `SimpleWriterExporterOutput`, no en una propiedad inventada de la configuración RTF. Esta decisión es coherente con el modelo de JasperReports: el exporter configuration decide opciones del formato; el writer decide cómo se codifican los caracteres al escribir texto.

La prueba automática busca la cabecera RTF. Es una comprobación sencilla pero significativa: evita aceptar un archivo vacío o un texto cualquiera con extensión `.rtf`. La ejecución completa confirma además que el exportador pudo recorrer el documento real, incluidos los componentes avanzados heredados de M5.

RTF se mantiene durante 6.5 aunque la configuración se centralice. La clase fábrica devuelve `SimpleRtfExporterConfiguration` y el generador conserva el writer UTF-8. Esto demuestra que refactorizar configuración no debe cambiar el contrato del archivo producido.

Profundización del bloque 4 — ODT como paquete OpenDocument

El reto ODT introduce `net.sf.jasperreports.engine.export.oasis.JR0dtExporter`. Un archivo ODT no es un flujo de texto simple: es un paquete ZIP con documentos XML, estilos y un archivo `mimetype` que identifica `application/vnd.oasis.opendocument.text`. Por eso el workflow valida el paquete como ZIP y comprueba ese `mimetype`.

El uso del paquete `oasis` en el nombre completo de la clase es relevante. Importar una clase desde un paquete distinto aunque tenga un nombre parecido provoca un fallo de compilación. El curso deja la referencia explícita en teoría, práctica y Java para evitar que el alumno copie una importación histórica o aproximada.

ODT cubre una necesidad distinta a RTF. Ambos pueden abrirse en procesadores de texto, pero ODT pertenece al estándar OpenDocument y empaqueta sus recursos de forma estructurada. En organizaciones que trabajan con LibreOffice o requieren formatos abiertos, puede ser una salida preferible.

La exportación sigue reutilizando el mismo `JasperPrint`. No se vuelve a ejecutar la base de datos para ODT. Esta constancia permite comparar resultados y mantiene alineados todos los formatos del módulo.

Profundización del bloque 5 — Orquestación multiformato, atonicidad y observabilidad

Generar varios formatos en una sola ejecución plantea una cuestión operativa: ¿qué sucede si el quinto exportador falla después de que los cuatro anteriores hayan creado sus archivos? El checkpoint utiliza una estrategia simple: cualquier excepción termina el proceso con código 1. Los archivos ya creados pueden permanecer en output, pero CI no considera la ejecución satisfactoria.

En un sistema productivo podría ser necesario un comportamiento más transaccional: escribir primero en un directorio temporal, validar todos los formatos y publicar el conjunto sólo cuando cada exportación haya terminado correctamente. Otra opción es permitir éxitos parciales y registrar un estado por formato. La política depende del contrato del servicio.

El curso utiliza logs de consola y artifacts de GitHub Actions como observabilidad básica. Cada ruta generada se imprime y el workflow publica los archivos incluso cuando un job falla, gracias al paso de artifacts con `if: always()`. Esto facilita diagnosticar un formato concreto sin perder la evidencia de los anteriores.

La matriz de contratos estructurales permite aplicar una prueba adecuada a cada formato. No tendría sentido verificar RTF buscando `%PDF-` ni validar CSV como ZIP. Diseñar pruebas específicas del formato es parte del aprendizaje del módulo, no un detalle auxiliar de CI.

También hay diferencias de semántica que la extensión del archivo no revela. CSV carece de tipos explícitos y depende del consumidor para interpretar números y fechas; XML posee estructura jerárquica; RTF y ODT buscan conservar propiedades de documento; PDF prioriza una representación paginada. La elección de formato debe partir del uso previsto y no de la idea de que todas las salidas son equivalentes.

Cuando un sistema entrega varios formatos al mismo usuario conviene documentar qué garantías ofrece cada uno. El PDF puede ser la copia oficial imprimible, XLSX el soporte de análisis, CSV el intercambio tabular, HTML la publicación navegable y ODT/RTF las versiones editables. El módulo introduce precisamente esa lectura funcional de la exportación multiformato.

Caso profesional — Elegir el formato por consumidor y no por costumbre

Una misma información puede tener destinatarios con necesidades incompatibles. Dirección puede querer un PDF estable para archivo; comercial, un XLSX con datos explotables; integración, un CSV sencillo; la intranet, HTML; un usuario de LibreOffice, ODT. Producir todos los formatos indiscriminadamente también tiene coste, por lo que el sistema debería conocer qué salidas necesita cada proceso.

La fidelidad debe definirse según el formato. Para PDF, fidelidad suele significar paginación y apariencia. Para XLSX, conservar valores, estructura y capacidad de cálculo puede ser más importante que la posición exacta. Para CSV, lo esencial es que delimitadores, codificación y registros sean inequívocos. Para XML, la estructura debe poder parsearse. Para RTF/ODT, el usuario espera un documento editable y razonablemente semejante al original.

Estos criterios permiten diseñar pruebas de aceptación más útiles. Un test que compare únicamente tamaños de archivo puede pasar aunque falten datos. Un test que busque una firma valida el contenedor pero no el contenido. El curso combina estructura del formato con invariantes del origen; en un proyecto real se añadirían valores de negocio concretos según el contrato de cada salida.

También conviene observar el tiempo y el consumo de memoria. Exportar siete formatos secuencialmente desde un mismo `JasperPrint` evita repetir el fill, pero añade trabajo de serialización y varios archivos. Un servicio con alta concurrencia puede decidir generar sólo formatos solicitados, cachear resultados o ejecutar determinadas conversiones de forma asíncrona.

El principio final es que multiformato no significa “copias idénticas en extensiones distintas”. Significa ofrecer representaciones coherentes del mismo estado de negocio, adaptadas a consumidores diferentes y verificadas con contratos apropiados.

Punto 6.5 — Configuración de exportación

Objetivos de aprendizaje

- Comprender el papel del objeto `JasperReportsContext` en la configuración global.
- Utilizar las propiedades del sistema para configurar el comportamiento de los exportadores.
- Crear un archivo `jasperreports.properties` con propiedades por defecto.
- Definir una clase de configuración personalizada para centralizar las opciones del proyecto.
- Combinar la configuración global con la configuración por exportador.
- Documentar la configuración de exportación del proyecto `EditorialReports`.

Bloque 1 — `JasperReportsContext` como contexto global del motor

`JasperReportsContext` es la interfaz que concentra propiedades y servicios compartidos por JasperReports. Cuando una operación no recibe un contexto específico, la biblioteca trabaja con el contexto por defecto. La implementación `DefaultJasperReportsContext` expone `getInstance()` y permite consultar o modificar propiedades mediante `getProperty` y `setProperty`. Para aislar cambios sin alterar globalmente el singleton puede utilizarse un `SimpleJasperReportsContext` cuyo padre sea el contexto por defecto.

JAVA

```
JasperReportsContext contexto = DefaultJasperReportsContext.getInstance();
String comprimido = contexto.getProperty(
    "net.sf.jasperreports.export.pdf.compressed");
```

Línea 1	obtiene el contexto compartido que JasperReports usa como referencia global cuando no se proporciona otro explícitamente.
Líneas 2-3	consulta desde ese contexto el valor efectivo de una propiedad de exportación PDF.

El contexto es importante porque separa el alcance global de la configuración específica de un exportador. Un valor leído por el contexto puede proceder de recursos del classpath o de propiedades inicializadas por el motor; después una configuración concreta como `SimplePdfExporterConfiguration` puede fijar explícitamente una opción para una exportación determinada.

En `EditorialReports` no se necesita crear un contexto personalizado para cada salida, porque las políticas específicas se expresan con las clases `Simple*Configuration`. Sin embargo, comprender `JasperReportsContext` explica de dónde proceden los valores globales y por qué `jasperreports.properties` debe llegar al classpath.

Bloque 2 — Propiedades del sistema y configuración externa a la aplicación

Las propiedades de sistema de Java permiten aportar valores a la JVM sin modificar el código fuente. Se pueden establecer al arrancar con la sintaxis `-Dclave=valor` o mediante `System.setProperty` antes de inicializar el comportamiento que dependa de ellas. Son útiles para parámetros de despliegue que deben cambiar entre entornos.

JAVA

```
System.setProperty(
    "net.sf.jasperreports.export.pdf.compressed",
    "true");
```

Líneas 1-3	establece en la JVM una propiedad antes de la exportación. Esta técnica tiene alcance de proceso y debe usarse con cuidado en aplicaciones que ejecutan muchos informes simultáneamente.
-------------------	--

En Jaspersoft Studio/Eclipse el mismo experimento puede realizarse desde **Run Configurations → Arguments → VM arguments** añadiendo una opción `-D...`. Para que el checkpoint final sea reproducible, M6 no depende de que el alumno conserve una opción manual del IDE: las decisiones permanentes quedan registradas en el repositorio mediante `jasperreports.properties` y las clases de configuración Java.

La distinción es pedagógicamente importante: una propiedad del sistema pertenece al entorno de ejecución; una propiedad de classpath pertenece al artefacto desplegado; una configuración de exportador pertenece a una operación concreta.

Bloque 3 — jasperreports.properties en el classpath

JasperReports puede cargar propiedades globales desde `jasperreports.properties` disponible en el classpath. El checkpoint 6.5 crea el fichero dentro de `EditorialReportsJava/src` y modifica Maven para copiar recursos no Java desde esa carpeta a `target/classes`.

PROPERTIES

```
net.sf.jasperreports.export.pdf.compressed=true
net.sf.jasperreports.export.csv.field.delimiter=;
```

Línea 1 define un valor global de compresión PDF.

Línea 2 define punto y coma como valor global asociado al delimitador CSV.

No basta con que el archivo exista visualmente en `src`. Si Maven no lo copia al classpath, el runtime no puede resolverlo como recurso. Por eso el `pom.xml` declara `src` como recurso y excluye `**/*.java`: las clases siguen compilándose normalmente y el `properties` se copia a `target/classes`.

El E2E valida exactamente esa premisa comprobando la existencia de `EditorialReportsJava/target/classes/jasperreports.properties` después de `mvn package`.

Bloque 4 — ConfiguraciónExportación como política reutilizable por formato

Las propiedades globales no sustituyen a las configuraciones específicas. `ConfiguracionExportacion.java` actúa como fábrica estática de políticas explícitas para PDF, XLSX, HTML, CSV y RTF. Los métodos reciben únicamente los valores variables, como título, autor o nombre de hoja.

JAVA

```
SimplePdfExporterConfiguration c =
    ConfiguracionExportacion.getConfiguracionPdf(
        "Informe de Ventas - EditorialReports",
        "Departamento Comercial");
```

PDF centraliza metadatos y compresión. XLSX mantiene separados `SimpleXlsxReportConfiguration` y `SimpleXlsxExporterConfiguration` porque representan niveles diferentes de la API. HTML centraliza cabecera, pie, CSS, separación entre páginas y conserva el reto Descargar PDF. CSV centraliza delimitadores y BOM. RTF centraliza su objeto de configuración, mientras UTF-8 permanece correctamente en `SimpleWriterExporterOutput`.

Esta separación reduce duplicación sin ocultar las clases reales de JasperReports. También permite que el generador se lea como una orquestación: llena el documento una vez y solicita una configuración reutilizable para cada salida.

Bloque 5 — Jerarquía, precedencia y cierre técnico del módulo

M6 combina tres niveles de configuración sin confundir sus alcances. Las propiedades de JVM son externas al artefacto; `jasperreports.properties` aporta valores globales desde el classpath; `JasperReportsContext` permite consultar o modificar propiedades en tiempo de ejecución; y las configuraciones de cada exportador fijan decisiones específicas para una operación concreta.

TEXT

```
Entorno JVM (-D / System.setProperty)
    |
    v
jasperreports.properties en classpath
    |
    v
JasperReportsContext
    |
    v
Simple*ReportConfiguration / Simple*ExporterConfiguration
    |
    v
exportReport()
```

El checkpoint final añade `ConfiguracionExportacion.java` y `jasperreports.properties`, modifica `pom.xml` para empaquetar recursos y refactoriza `GeneradorInformeVentas.java`. No modifica el JRXML ni el JRTX cerrado en M5.

La validación vuelve a generar PDF normal/protegido, XLSX de ventas y catálogo, HTML con enlace al PDF, CSV, XML, RTF y ODT. El E2E comprueba además que la clase central se usa realmente y que el `properties` está en `target/classes`. Los invariantes continúan siendo 14 libros, 9 ventas, 31 unidades, 633,40 € y seis páginas.

Así se cubren los seis objetivos originales: comprender el contexto, utilizar propiedades de sistema como mecanismo de entorno, crear propiedades por defecto, centralizar opciones, combinar niveles de configuración y documentar el resultado.

Profundización del bloque 1 — Alcance de la configuración y responsabilidad arquitectónica

Centralizar configuración no significa trasladar toda la aplicación a una clase estática. El objetivo es reunir decisiones repetitivas que pertenecen a la política de exportación: compresión PDF, metadatos comunes, opciones XLSX, cabecera HTML, delimitadores CSV o configuración RTF. Las rutas de archivos, los datos y el orden de ejecución siguen perteneciendo al generador.

Esta separación reduce duplicación y hace visibles los puntos de variación. Si cambia el autor corporativo de los PDFs, existe un lugar claro para modificarlo. Si el XLSX debe mostrar cuadrícula en otro proyecto, se cambia la fábrica correspondiente sin tocar la lógica de fill. El beneficio aumenta cuando varios informes comparten la misma política.

Una clase de métodos estáticos es suficiente para el alcance docente, pero no es la única arquitectura posible. En una aplicación mayor podría inyectarse un servicio de configuración, cargar propiedades externas o construir perfiles por cliente. Lo importante es separar responsabilidades y mantener los tipos reales de JasperReports en lugar de ocultarlos detrás de una abstracción que impida usar capacidades específicas.

El E2E busca expresamente llamadas a `ConfiguracionExportacion`. De ese modo se demuestra que la clase central no es un archivo decorativo añadido al árbol: el generador la utiliza de verdad.

Profundización del bloque 2 — Jerarquía de configuración y precedencia

JasperReports admite valores procedentes de distintas capas: propiedades disponibles en el contexto, archivos del classpath y configuraciones pasadas directamente al exportador. Cuanto más específica es la configuración, más fácil resulta razonar sobre una exportación concreta; cuanto más global es, menos código se repite.

`jasperreports.properties` resulta apropiado para defaults transversales que deben estar disponibles para la librería. La clase `ConfiguracionExportacion` es apropiada para políticas del proyecto que se construyen con valores dinámicos, como el título o el nombre de hoja. El generador decide qué configuración aplica a cada salida.

No conviene asumir una regla de precedencia sin verificar la propiedad concreta y la versión de la librería. Por eso el curso no intenta trasladar todas las opciones a `jasperreports.properties`. Sólo coloca defaults sencillos y mantiene explícitas en Java las decisiones que forman parte del ejercicio.

La separación XLSX sigue siendo un buen ejemplo: el método `getConfiguracionXlsxReport` devuelve un `SimpleXlsxReportConfiguration` y `getConfiguracionXlsxExportador` devuelve un `SimpleXlsxExporterConfiguration`. La centralización no borra la distinción de ámbitos estudiada en 6.2.

Profundización del bloque 3 — Classpath, Maven resources y por qué “tener el archivo” no basta

JasperReports busca `jasperreports.properties` en el classpath. Colocarlo en el repositorio no garantiza que esté disponible durante la ejecución. El proyecto de este curso utiliza `src` como carpeta de fuentes Java; por eso el `pom.xml` añade una sección `resources` que copia los archivos no Java desde esa carpeta a `target/classes`.

Este detalle es un buen ejemplo de trazabilidad entre construcción y runtime. El código Java puede compilar aunque el `properties` no se copie. El fallo aparecería sólo cuando se esperara que JasperReports leyera esos defaults. El E2E evita esa ambigüedad comprobando físicamente `target/classes/jasperreports.properties` después de `mvn package`.

El mismo principio se aplica a otros recursos: plantillas, imágenes, mensajes o certificados deben estar disponibles en la ruta que el runtime realmente utiliza, no sólo en una carpeta que resulta cómoda para el desarrollador.

En proyectos Maven convencionales se utilizaría normalmente `src/main/java` y `src/main/resources`. EditorialReports conserva la estructura simplificada heredada del curso y adapta el build de forma explícita. La práctica explica el porqué para que el alumno pueda trasladar la idea a estructuras estándar.

Profundización del bloque 4 — Configuración dinámica, entornos y secretos

No todas las opciones deberían codificarse como constantes. El título documental o el nombre de hoja pueden depender del informe; rutas y políticas pueden variar entre desarrollo y producción. Contraseñas de usuario o propietario, en particular, no deberían quedar embebidas en código fuente en una aplicación real.

El checkpoint mantiene credenciales conocidas únicamente porque necesita una demostración reproducible de cifrado. En producción, esos valores deberían llegar desde un almacén de secretos, variables de entorno o configuración segura. Centralizar la construcción de configuraciones facilita introducir esa fuente externa más adelante sin modificar cada exportador.

Las propiedades del sistema de Java ofrecen otro mecanismo de entorno. Un proceso puede arrancar con flags `-D...` y JasperReports puede consultar determinados valores a través de su contexto. Esta técnica es útil para defaults operativos, aunque debe documentarse para evitar que una aplicación cambie de comportamiento de forma invisible.

La regla práctica es separar aquello que define el código del producto de aquello que define el despliegue. El módulo muestra ambos mecanismos, pero no obliga a concentrarlos todos en el mismo nivel.

Profundización del bloque 5 — Refactorización segura y pruebas de regresión

Una refactorización se considera correcta cuando cambia la estructura interna sin cambiar el comportamiento observable que debe conservarse. En 6.5 se mueve configuración desde `GeneradorInformeVentas` hacia `ConfiguracionExportacion` y se incorpora `jasperreports.properties` al classpath. Los formatos que funcionaban en 6.4 deben seguir funcionando.

Por eso el E2E de 6.5 no se limita a compilar la nueva clase. Regenera PDF normal/protegido, XLSX de ventas y catálogo, HTML con CSS y enlace al PDF, CSV, XML, RTF y ODT. También conserva los invariantes de datos y las seis páginas del JasperPrint. Cualquier pérdida causada por la refactorización aparece como una regresión.

El reto HTML fue especialmente útil: una primera centralización podía haber reemplazado una cabecera personalizada y perder el enlace [Descargar PDF](#). La prueba final exige que el enlace continúe presente dentro de la configuración central. Este tipo de caso demuestra por qué las pruebas deben cubrir requisitos y no sólo clases.

La cadena acumulativa ofrece una segunda defensa. `audit_m6_traceability.py` conoce qué archivos puede añadir o modificar cada punto. Si 6.5 eliminara un recurso heredado o cambiara el JRXML, el módulo dejaría de cumplir su contrato aunque los exportadores siguieran compilando.

El resultado final es una arquitectura más mantenible sin sacrificar evidencia. Configuración, código, properties, outputs, documentación y tests describen el mismo estado y pueden reconstruirse desde el repositorio.

Caso profesional — Evolucionar la política de exportación sin romper informes

Una ventaja práctica de la centralización aparece cuando cambia una norma corporativa. Imaginemos que todos los PDFs deben incluir un nuevo creador documental, que los XLSX deben mostrar cuadrícula o que el separador CSV cambia para una integración concreta. Si cada generador configura sus exportadores de forma independiente, localizar todas las variantes es difícil y es fácil dejar un informe con la política anterior.

`ConfiguracionExportacion` ofrece un punto de evolución controlado. Un cambio en la fábrica se propaga a los consumidores que la utilizan, pero precisamente por eso necesita pruebas de regresión. Una modificación aparentemente pequeña puede afectar a todos los informes de una aplicación. La centralización reduce duplicación; no elimina la necesidad de validar.

Los valores dependientes del entorno deberían mantenerse fuera de la clase cuando corresponda. Una contraseña, una ruta de publicación o una opción experimental pueden recibirse desde variables de entorno o un sistema de configuración. La fábrica puede transformar esos valores en objetos JasperReports sin convertirse en almacén de secretos.

También es útil probar la configuración de forma aislada. Un test unitario podría llamar a `getConfiguracionPdf` y comprobar sus propiedades; otro podría verificar que `getConfiguracionXlsxReport("Ventas")` conserva el nombre de hoja esperado. Esos tests serían rápidos y complementarían el E2E, que sigue siendo imprescindible para saber que JasperReports interpreta las opciones y genera archivos reales.

El patrón estudiado en 6.5 es, por tanto, una pequeña aplicación de diseño de software: separar política de orquestación, mantener configuración global cuando aporta valor, permitir excepciones específicas y proteger la refactorización con pruebas que observan el comportamiento final.